String:

String in C programming is a sequence of characters terminated with a null i.e character '\0'. Strings are defined as an array of characters. The difference between a character array and a string is the string is terminated with a unique character '\0'.

Or String is a collection of characters or it's an array of character.

Declaration of Strings

Declaring a string is as simple as declaring a one-dimensional array. Below is the basic syntax for declaring a string.

char str_name[size];

In the above syntax str_name is any name given to the string variable and size is used to define the length of the string, i.e the number of characters strings will store.

Note: There is an extra terminating character which is the *Null character ('\0') used to indicate the termination of a string that differs strings from normal character arrays*. When a Sequence of characters enclosed in the double quotation marks is encountered by the compiler, a null character '\0' is appended at the end of the string by default.

Initializing a String

A string can be initialized in different ways. We will explain this with the help of an example. Below are the examples to declare a string with the name str and initialize it with "Hello".

4 Ways to Initialize a String in C

1. Assigning a string literal without size:

String literals can be assigned without size. Here, the name of the string str acts as a pointer because it is an array.

char str[] = "Hello";

2. Assigning a string literal with a predefined size:

String literals can be assigned with a predefined size. But we should always account for one extra space which will be assigned to the null character. If we want to store a string of size n then we should always declare a string with a size equal to or greater than n+1.

char str[10] = " Hello";

3. Assigning character by character with size:

We can also assign a string character by character. But we should remember to set the end character as '\0' which is a null character.

char str[6] = { 'H','e','l','l','o','\0'};

4. Assigning character by character without size:

We can assign character by character without size with the NULL character at the end. The size of the string is determined by the compiler automatically.

char str[] = { 'H','e','l','l','o','\0'};

Below is the memory representation of the string “Hello”.

	0	1	2	3	4	5
str	H	e	l	l	o	\0

Simple program to get a clear understanding of declaring, initializing a string in C, and also how to print a string with its size.

```
/ C program to read string from user
#include<stdio.h>

int main()
{
   // declaring string
   char str[50];

   // reading string
   printf(“\n Eenter the string:);
   scanf("%s",str);

   // print string
  printf(“\n Eentered string is:);
   printf("%s",str);

   return 0;
}
```

Output:

Eenter the string: Hello

Eentered string is: Hello

Operation on String:

String Manipulation built-in or standard library functions:

All string manipulation standard functions predefined in string.h header file.

1. Strcpy(): used to copy a string to another string.

Syntax:

```
char *strcpy(char * str1, const char *str2);
```

here it copies data from str2 to str1.

Example:

```
#include <stdio.h>
 #include <string.h>


 int main() {
   char str1[20] = "C programming";
   char str2[20];


   // copying str1 to str2
   strcpy(str2, str1);


   puts(str2); // C programming


   return 0;
 }
```

Output:

C programming

2. Strcat(): used to concatenate two strings.

Syntax:

```
char *strcat(char * str1, const char *str2);
```

here it appends the data of str2 to str1.

```
#include <stdio.h>
#include <string.h>
int main() {
  char str1[100] = "MMEC ", str2[] = "BELAGAVI";

  // concatenates str1 and str2
  // the resultant string is stored in str1.
  strcat(str1, str2);

  puts(str1);
  puts(str2);

  return 0;
}
```

Output

```
MMECBELAGAVI
BELAGAVI
```

3. Strlen(): used to find the length of string.

Syntax:

Size_t strlen(const char *str);

Here it finds the length of str.

Example:

```
#include <stdio.h>
#include <string.h>
int main()
{
   char a[20]="Program";
   char b[20]={'P','r','o','g','r','a','m','\0'};

   // using the %zu format specifier to print size_t
   printf("Length of string a = %zu \n",strlen(a));
   printf("Length of string b = %zu \n",strlen(b));

   return 0;
```

```
}
```

Output:
Length of string a = 7
Length of string b = 7

4. Strstr(): used to find the sub string of given string.

Syntax:

```
char *strstr(const char * str1, const char *str2);
```

here it checks whether str2 is sub string of str1 or not.

```
#include <string.h>
#include <stdio.h>
 int main()
{
   char s1[] = "Programming in c";
   char s2[] = "Programming";
   char* p;

   // Find first occurrence of s2 in s1
   p = strstr(s1, s2);

   // Prints the result
   if (p) {
      printf("String found\n");
      printf(""%s' is substring of '%s' ", s2, s1);
   }
   else
```

```
        printf("String not found\n");
 return 0;
}
```

Output:

String found

'Programming' is substring of 'Programming in c'

5. Strcmp(): used to compare two strings.

Syntax:

char *strcmp(const char * str1, const char *str2);

here it compare whether str1 and str2 are same or not.

```
#include <stdio.h>
#include <string.h>
int main() {
  char str1[] = "mmec", str2[] = "mmec";
  int result;
 result = strcmp(str1, str2);
if(result==0)
  printf("\n %s and %s are equal”);
else
      printf("\n %s and %s are equal”);
   return 0;
}
```

Output:

mmec and mmec are equal

Character manipulation standard library functions.

C provides standard character handling library <ctype.h>.

This header include various useful functions for performing tests and manipulations of character data in C programming.

islower(ch);

It returns a true if ch is a lowercase letter and 0 otherwise.

isupper(ch);

It returns a true value if ch is a uppercase and 0 therwise.

tolower(ch);

This function converts uppercase letter into lowercase letter.

toupper(ch);

This function converts lowercase letter into uppercase letter.

isblank(ch);

This function checks whether ch is a blank character or not.

isdigit(ch);

This function checks whether ch is a digit or not.

isalpha(ch);

This function checks whether ch is a letter or not.

isalnum(ch);

This function returns a 1 if ch is a digit or a letter and 0 otherwise.

isxdigit(ch);

This function checks if ch is a hexadecimal digit or not.

isspace(ch);

This function checks if ch is a whitespace character or not.

Simple Examples for isalpha():

```
#include <stdio.h>
```

```
#include <ctype.h>
int main()
{
    char c;

    printf("Enter a character: ");
    scanf("%c", &c);

    if (isalpha(c) == 0)
        printf("%c is not an alphabet.", c);
    else
        printf("%c is an alphabet.", c);

    return 0;
}
```

Simple Examples for isdigit ()

```
#include <stdio.h>
#include <ctype.h>

int main()
{
    char c;

    printf("Enter a character: ");
    scanf("%c",&c);
```

```
    if (isdigit(c) == 0)
        printf("%c is not a digit.",c);
    else
        printf("%c is a digit.",c);
    return 0;
}
```

Simple Examples for isalnum()

```
#include <stdio.h>
#include <ctype.h>
int main()
{
    char c;
    printf("Enter a character: ");
    scanf("%c", &c);

    if (isalnum(c) == 0)
        printf("%c is not an alphanumeric character.", c);
    else
        printf("%c is an alphanumeric character.", c);

    return 0;
}
```

Array of Strings:

Array of strings in C is a two-dimensional array of character types. Each String is terminated with a null character (\0). It is an application of a 2d array.

char variable_name[r][c] = {list of string};

Here,

- variable_name is the name of the variable in C.
- r is the maximum number of string values that can be stored in a string array.
- c is a maximum number of character values that can be stored in each string array.

Example:

```
char name[3][5];
```

In the above declaration, name is array of strings which can stores maximum up to 3 names with each name length up to 5 characters.

```
#include <stdio.h>
int main()
{
    int i;
        char arr[3][10] = {"abc", "tuvw", "xyz"};
        printf("String array Elements are:\n");

        for (i = 0; i < 3; i++)
        {
            printf("%s\n", arr[i]);
        }
        return 0;
}
```

Output:

```
String array Elements are:
abc
```

tuvw

xyz

Memory representation of array of strings:

	0	1	2		3	4	5
arr[0]	a	b	c	\0			
arr[1]	t	u	v	w	\0		
arr[2]	x	y	z	\0			

Pointers:

Introduction to pointers

Pointer is special type of variable, which stores the address of another variable.

A pointer is a derived data type in C that can store the address of other variables or a memory. We can access and manipulate the data stored in that memory location using pointers.

Pointers in C are used to store the address of variables or a memory location. This variable can be of any data type i.e, int, char, function, array, or any other pointer. Pointers are one of the core concepts of C programming language that facilitates dynamic memory allocation.

Syntax:
datatype * *pointer_name*;

How to Use Pointers?

1. Addressof Operator

The addressof operator (&) is a unary operator that returns the address of its operand. Its operand can be a variable, function, array, structure, etc.

Syntax of Address of Operator
&*variable_name*;

2. Dereferencing Operator

The dereference operator (*), also known as the indirection operator is a unary operator. It is used in pointer declaration and dereferencing.

Declaration:

*(star) operator is used to declare the variable as pointer.

int *a;

float *f;

In the above declaration, 'a' is pointer variable of type integer and 'f' is the pointer variable of type float.

Pointer Initialization:

Initialization of pointers can be done using two different methods.

Method 1: C Pointer Definition

datatype * pointer_name = *address*;
The above method is called Pointer Definition as the pointer is declared and initialized at the same time.

Example: int a;

int *ptr=&a;

Method 2: Initialization After Declaration

The second method of pointer initialization in C the assigning some address after the declaration.

datatype * pointer_name;

pointer_name = *addresss*;

example:

Since the pointer stores the address of the another variable, need to declare the normal variable first and then assign the address of the variable to pointer.

```
int x;

flaot y;

int *a;

float *f;

a=&x;      // pointer x is initialized with address of variable x.

f=&y;      // pointer y is initialized with address of variable y.
```

Note: pointer stores the address of another variable of same type. Means the data type of the pointer variable and the address of the variable which is going to be initialized should be same.

a=&f // wrong initialization because the pointer variable 'a' is integer type and variable 'f' is floating type.

Types of pointers:

Null pointer:

a null pointer is a pointer that does not point to any memory location and hence does not hold the address of any variables. It just stores the segment's base address. That is, the null pointer in C holds the value Null.

A null pointer is a special reserved value declared in a header file called stdio.h.

Syntax

The syntax of a null pointer can be defined in two ways.

```
int *pointer_var = NULL;

int *pointer_var = 0
```

A Null pointer can be used in the following ways:

- When the pointer variable does not point to a valid memory address, it is used to initialize it.
- Before dereferencing pointers, it is utilized to execute error handling.
- When we don't want to pass the memory location directly, we pass it as a function argument, which is then utilized to return from a function.

Void or generic pointer:

The void pointer in C is a pointer which is not associated with any data types.

```
#include<stdlib.h>

int main() {

   int a = 7;

   float b = 7.6;

   void *p;

   p = &a;

   printf("Integer variable is = %d", *( (int*) p) );

   p = &b;

   printf("\nFloat variable is = %f", *( (float*) p) );

   return 0;

}
```

Output:

Integer variable is = 7
Float variable is = 7.600000

Passing arguments to functions using pointers

```
#include <stdio.h>



void swap(int *x,int *y)

{

    int t;

     t   = *x;

    *x   = *y;

    *y   =  t;
```

```
}


int main()
{
    int num1,num2;


    printf("Enter two numbers: ");
    scanf("%d%d",&num1,&num2);
    printf("Before Swapping:\n num1 is: %d \n num2 is: %d\n",num1,num2);
    swap(&num1,&num2);
    printf("After  Swapping:\n num1 is: %d \n num2 is: %d\n",num1,num2);
    return 0;
}
```

Output:

```
Enter two numbers: 1     2
Before Swapping:
 num1 is: 1
 num2 is: 2
After  Swapping:
 num1 is: 2
 num2 is: 1
```